# Lecture 09: Python Basics (Chapter 6)

교재 Ch. 6.4, 6.5, 6.6

유준수 (Joon Soo Yoo, Ph.D.)

2026년 3월 24일

방산첨단보안연구실 (DAS Lab)
전북대학교 첨단방위산업학과

# 6.4 Parameters & Mutability

함수로 데이터 전달하기와 가변성의 비밀

# 매개변수(Parameter) vs 인자(Argument)

두 용어는 혼용되기도 하지만, 엄밀히는 위치에 따라 구분됩니다.

- **매개변수(Parameter):** 함수 **정의** 시 사용하는 변수 이름 (붕어빵 틀의 구멍)
- **인자(Argument):** 함수 **호출** 시 실제로 전달하는 값 (붕어빵 반죽)

## 예시 코드

```
def greet(name):    # name: Parameter
    print(f"Hello {name}")
user = "JoonSoo"

greet(user)    # user: Argument
```

# 참고 (1): f-string (Ch. 3.2)

**f-string**은 변수와 문자열을 결합하는 가장 빠르고 읽기 쉬운 방법입니다. 문자열 앞에 소문자 `f`를 붙이고, 변수나 식을 중괄호 {} 안에 넣으면 됩니다.

### 기본 사용법 (Interpolation)

```
name = "Joon"; age = 35
print(f"내 이름은 {name}이고, 내년에는 {age + 1}살입니다.")
# 출력: 내 이름은 Joon이고, 내년에는 36살입니다.
```

- **직관성:** 변수명을 문자열 안에 직접 써서 한눈에 이해됩니다.
- **강력함:** 중괄호 안에서 덧셈, 곱셈, 함수 호출 등 **모든 파이썬 연산**이 가능합니다.

# 참고 (2): f-string 실전 활용 예시

## 1. 산술 연산 및 문자열 메서드

```
x = 10; y = 20; name = "das lab"
print(f"{x} + {y} = {x + y}")                    # 출력: 10 + 20 = 30
print(f"Welcome to {name.upper()}!")             # 출력: Welcome to DAS LAB!
```

## 2. 소수점 제어 및 포맷팅 (연구 데이터 출력 시 필수)

```
accuracy = 0.95876; price = 1250000
print(f"정확도: {accuracy:.2}%")                 # 출력: 정확도: 95.88%
print(f"비트코인: ${price:,}")                   # 출력: 비트코인: $1,250,000
```

# 다중 매개변수: 순서가 생명!

함수는 여러 개의 입력값을 받을 수 있습니다. 이때 인자를 전달하는 **순서**가 매우 중요합니다.

## 나눗셈 함수 예시

```
def print_div(op_1, op_2):
    print(f"{op_1} / {op_2} = {op_1/op_2}")
print_div(6, 3)    # 결과: 6 / 3 = 2.0
print_div(3, 6)    # 결과: 3 / 6 = 0.5
```

## 주의사항

매개변수의 개수와 인자의 개수가 맞지 않으면 `TypeError`가 발생합니다.

# 가변성(Mutability)과 인자 수정

파이썬은 **객체 참조 전달(Pass-by-object-reference)** 방식을 사용합니다. 함수에 인자를 넘길 때, 객체의 '주소'를 넘겨주는 방식입니다.

**가변성에 따른 차이**

| 유형 | 불변 객체 (Immutable) | 가변 객체 (Mutable) |
|---|---|---|
| **종류** | 정수, 실수, 문자열, 튜플 | **리스트**, 딕셔너리, 세트 |
| **함수 내 수정** | 원본은 변하지 않음 (복사본 개념) | **원본이 함께 변함** (공유 개념) |

## 6.4.3 불변 객체의 전달 (Immutable: Integer)

정수(int)는 불변 객체입니다. 함수 안에서 값을 변경하려고 하면, 파이썬은 기존 값을 고치는 게 아니라 **새로운 숫자 객체**를 만들어 할당합니다.

### 정수 전달 예시

```
def update_val(n):
    n += 10    # 새로운 객체 생성 (기존 n과는 별개)
    print(f"함수 내부: {n}")
num = 5

update_val(num)
print(f"함수 외부: {num}")                    # 여전히 5!
```

## 6.4.4 가변 객체의 전달 (Mutable: List)

리스트(list)는 가변 객체입니다. 함수에 리스트를 전달하면 주소값을 공유하므로, 함수 내부의 수정이 **원본 데이터**를 직접 바꿉니다.

### 리스트 전달 예시

```
def add_score(scores):
    scores.append(100)    # 주소지에 직접 데이터 추가
    print(f"함수 내부: {scores}")
my_list = [80, 90]

add_score(my_list)
print(f"함수 외부: {my_list}")                    # [80, 90, 100]으로 바뀜!
```

# 예시 1: 삼각형 넓이 계산기 (Parameters)

**Mission: 밑변과 높이를 받아 넓이를 출력하는 함수를 만드세요.**

1. `print_area(base, height)` 함수를 정의합니다.
2. 삼각형 넓이 공식($A = 1/2 \times b \times h$)을 사용하여 결과를 계산합니다.
3. "Triangle area: [결과]" 형식으로 출력하세요.

# 6.5 Return Values

함수의 결과물을 변수에 담기 (반환값)

# Return: 함수의 최종 응답

함수가 실행을 마치고 자신을 호출한 곳으로 결과를 전달하는 명령입니다.

- **반환(Return):** 결과값을 가지고 호출 지점으로 돌아감.
- **종료:** `return` 문을 만나는 즉시 함수의 실행은 끝납니다.

### print() vs return 의 차이

**[print]** 화면에 "보여만" 줄 뿐, 그 값을 변수에 저장할 수는 없음.

**[return]** 값을 "전달"하므로, 다른 계산에 재사용하거나 변수에 담을 수 있음.

### None: 아무것도 돌려주지 않을 때

`return` 문이 없거나 값 없이 `return`만 쓰면 파이썬은 자동으로 `None`을 반환합니다.

, shrink=5다중 Return 문 (Multiple Return) 조건문에 따라 서로 다른 값을 반환해야 할 때 여러 개의 `return`을 사용할 수 있습니다.

### 안전한 연비 계산기 (calc_mpg)

```
def calc_mpg(miles, gallons):
    if gallons > 0:
        return miles / gallons   # 정상 계산 시 반환
    else:
        print("Error: Zero gallons")
        return -1   # 오류 발생 시 특수값 반환
```

- **보안 팁:** 오류 상황에서 `-1`이나 `False`를 반환하여 프로그램이 멈추지 않게 방어적으로 코딩하는 습관이 중요합니다.

# 함수를 값으로 활용하기 (Function Composition)

함수의 반환값은 그 자체로 하나의 **값**이므로, 다른 함수의 인자로 넣거나 수식에 섞어 쓸 수 있습니다.

## 함수 합성 예시

```
def sq(n): return n * n
def offset(n): return n - 2
 val = sq(offset(5))   # offset(5)는 3을 반환 -> sq(3) 실행

print(val)   # 결과: 9
```

# 예시 2: 생존 일수 계산기 (Estimated Days Alive)

**Mission: 나이를 입력받아 살아온 일수를 반환하는 함수를 만드세요.**

1. `days_alive(age)` 함수를 작성합니다.
2. 공식: $age \times 365$
3. 메인 프로그램에서 사용자 나이를 입력받아 위 함수를 호출하고 결과를 출력하세요.

**출력 예시**

```
Input: 21 → You have been alive about 7670 days.
```

# 6.6 Keyword Arguments

인자 전달의 유연성과 기본값 설정

# 위치 인자 vs 키워드 인자

함수를 호출할 때 값을 전달하는 방식은 두 가지가 있습니다.

**1. 위치 인자 (Positional)**
순서에 따라 매개변수에 할당됨.

```
greeting("Hi", "User", 3)
```

→ **순서가 중요!**

**2. 키워드 인자 (Keyword)**
이름을 지정하여 값을 할당함.

```
greeting(count=3, msg="Hi",
name="User")
```

→ **순서 상관 없음!**

**팁: 위치 인자가 무조건 먼저!**

위치 인자와 키워드 인자를 섞어 쓸 때는, **위치 인자가 반드시 앞에** 와야 합니다.

`greeting("Hi", count=3, name="User")` **(O)**

`greeting(count=3, "Hi", "User")` **(X - Error)**

# 예시: 위치 인자와 키워드 인자 섞기

### 함수 정의 (Greeting)

```
def greeting(msg, name, count):
    for _ in range(count):
        print(f"{msg}, {name}!")
```

## 올바른 호출 방법 (Correct Usage)

- **위치 전용:** `greeting("안녕", "철수", 1)` **(OK)**
- **키워드 전용:** `greeting(count=1, name="영희", msg="반가워")` **(OK)**
- **혼합 사용:** `greeting("반가워", count=2, name="민수")` **(OK)**

### SyntaxError 주의! (Bad Case)

```
greeting(count=2, "반가워", "민수")
# 에러: positional argument follows keyword argument
```

**(Error!)**

# 매개변수 기본값 (Default Values)

인자가 전달되지 않았을 때 사용할 '기본값'을 미리 정해둘 수 있습니다.

### 기본값이 있는 함수 정의

```
def greeting(msg, name="Friend", count=1):
    for i in range(count):
        print(msg, name)
```

- `greeting("Hello")` → "Hello Friend" 출력 (기본값 사용)
- `greeting("Hi", name="Gina")` → "Hi Gina" 출력

### 주의: 가변 객체(Mutable)는 기본값으로 금지!

리스트(`[]`)나 딕셔너리 같은 가변 객체를 기본값으로 쓰면 예기치 못한 버그가 생깁니다. 항상 `None`이나 불변 객체를 사용하세요.

# PEP 8 스타일 가이드: 띄어쓰기

파이썬 공식 스타일 가이드(PEP 8)는 키워드 인자나 기본값을 지정할 때 **등호(=) 주변에 공백을 두지 않을 것**을 권장합니다.

**올바른 표기법**

| Good (권장) | Bad (비권장) |
|---|---|
| `donate(amount=10)` | `donate(amount = 10)` |
| `def func(n=5):` | `def func(n = 5):` |

*"작은 습관이 가독성 좋은 코드를 만듭니다."*

# 예시 3: 스트리머 후원 시스템 (Stream Donations)

**Mission: 후원 금액, 이름, 메시지를 처리하는 함수를 만드세요.**

1. `donate()` 함수를 정의합니다.
2. 매개변수와 기본값: `amount=5`, `name="Anonymous"`, `msg=""`
3. 출력 형식: "[이름] donated [금액] credits: [메시지]"
4. **실습:** 위치 인자와 키워드 인자를 각각 사용하여 호출해 보세요.

**호출 예시**

```
donate(10, msg="gg")
```

→ `Anonymous donated 10 credits: gg`

## Any Questions?